Software Requirements Specification (SRS)

System: Jima — CARAVAN Lounge Restaurant Management System
Document: SRS v1.3 · **Updated:** August 30, 2026
Standard: Structured after IEEE 830

1. Introduction

1.1 Purpose

This SRS specifies the complete requirements — functional and non-functional — for the Jima Restaurant Management System serving CARAVAN Lounge. It is intended for developers, testers, and stakeholders taking over or extending the system.

1.2 Scope

The system digitizes end-to-end restaurant operations: table-side ordering (POS), kitchen workflow, cashiering, inventory and procurement, staff and branch administration, and management analytics. It is a web application accessed from browsers on desktops, tablets, and phones.

1.3 Definitions

Term	Meaning
POS	Point of Sale — the screen waiters use to compose orders.
Order Board	Live kanban-style view of order progress.
JWT	JSON Web Token used for stateless authentication.

Item Request	The older generic request against branch inventory, with its own approval/issue lifecycle.
Branch Stock Request	A separate request by a Branch Storekeeper for Main Store stock to be transferred to that storekeeper's assigned destination branch.
Main Store Storekeeper	A storekeeper assigned to a restaurant but with no branch assignment; the only actor permitted to fulfill an approved Branch Stock Request.

2. Overall Description

2.1 Product Perspective

Three-tier logical architecture: a **Next.js 15 / React 19 frontend**, a **NestJS REST API backend**, and a **PostgreSQL database** accessed through TypeORM. The registered Jima web artifact runs the actual source in `artifacts/nextjs-app`. The frontend calls the backend through `/backend/api/*`; Next.js rewrites requests to the backend's `/api/*`. The separate `artifacts/api-server` is not the restaurant API.

2.2 User Classes

Eight roles: admin, owner, manager, coordinator, waiter, chef, cashier, storekeeper (see FRS §2). Users are non-technical restaurant staff; the UI must remain simple, large-touch-friendly, and in plain language.

2.3 Operating Environment

Layer	Technology
Frontend	Next.js 15 (App Router), React 19, TypeScript, Tailwind CSS, Zustand (auth store), Recharts, Axios
Backend	NestJS 10, TypeScript, Passport-JWT, bcrypt, TypeORM
Database	PostgreSQL 14+ (TypeORM `synchronize: true` in development)
Runtime	Node.js 18+, pnpm monorepo workspaces

2.4 Constraints

- Protected API access requires JWT. Login and health are public; seed operations are administrative.
- The current backend still enables TypeORM schema synchronization. Production is blocked until synchronization is disabled there and a tested migration/rollback baseline exists.
- Realtime behavior is implemented by 10-second polling, not WebSockets.
- Currency is Ethiopian Birr (ETB); demo data uses Ethiopian dishes and names.

3. Functional Requirements

The complete functional catalogue is maintained in the FRS (document 01). Summary of subsystems:

Subsystem	Capabilities
Auth	Login, JWT issuance, role-based menus and guards
Orders / POS	Create, append items, confirm + assign chef, status lifecycle, filters, auto-refresh
Kitchen	Chef queues, accept/preparing/ready, live progress board with delay flags
Payments	Cash/card/wallet payment capture, change, daily report, shifts
Menu	Categories and items with availability
Inventory	Branch stock, central Main Store stock, low-stock alerts, suppliers, purchase orders, adjustments, generic Item Requests, and the separate Branch Stock Request / Main Store transfer workflow
Admin	Staff, restaurants, branches, tables
Analytics	Manager/owner one-page dashboard, exportable reports (Excel/PDF), cashier-scoped reports
Notifications	Role/user-targeted messages with read tracking

3.1 Branch Stock Request / Main Store requirements

Stage	Required behavior
Request	Only a storekeeper with a branch assignment may request positive quantities of Main Store items. The destination is derived from the requester's assigned branch, and both items and destination must belong to the requester's restaurant. Creation records the requester and creation timestamp but does not reserve, decrement, or increment stock.
Decision	Only the destination branch's Manager or an Owner in the same restaurant may approve or reject a pending request. The system records the approver and approval time or rejector and rejection time. Neither decision changes stock.
Fulfillment	Only a branchless Main Store Storekeeper in the same restaurant may fulfill an approved request. In one transaction, the system locks and validates the request, destination, lines, and Main Store quantities; decrements Main Store; and increments or creates the matching name-and-unit inventory item in the assigned destination branch. Any error rolls back every line.
Audit	Each fulfilled line links a Main Store stock-out movement and destination-branch addition adjustment to the transfer, records the fulfilling actor, and preserves both post-transfer balances. The request also records requester, decision actor, fulfillment actor, and corresponding timestamps.
Isolation	Every operation is restaurant-scoped. Branch Storekeepers and Managers may list only their assigned destination branch's transfers; a Manager cannot decide another branch's request; fulfillment cannot use another restaurant's Main Store item or destination branch.
Separation	The older generic Item Requests workflow remains a separate supported workflow. Branch Stock Requests do not replace it and must use distinct lifecycle, permissions, records, and stock effects.

4. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Standard pages should render within 2 s on a typical broadband connection; list endpoints return within 500 ms for up to 10,000 orders.
NFR-02	Freshness	Operational views (orders, kitchen, boards, dashboards) auto-refresh every 10 seconds without flicker.
NFR-03	Security	JWT-based auth; bcrypt password hashing; role guards on every protected endpoint; secrets provided via environment variables, never committed.
NFR-04	Usability	Plain-language UI, color-coded statuses, and touch-friendly controls. The Orders/POS flow must remain usable from a 320 px phone viewport upward: no overlapping checkout controls, intentional horizontal scrolling only for category tabs or data tables, and reachable checkout actions through vertical scrolling.
NFR-05	Reliability	Backend failures surface explicit errors; the frontend degrades gracefully (empty states, retry on next poll).
NFR-06	Maintainability	TypeScript end-to-end; modular NestJS feature modules; monorepo with shared conventions; `tsc --noEmit` must pass.
NFR-07	Portability	Runs anywhere Node 18+ and PostgreSQL are available; deployable to Replit, Vercel (frontend) + managed Node host (backend), or a single VM.
NFR-08	Auditability	Stock adjustments and requests record acting users and timestamps. Branch Stock Request evidence additionally links request, decision, fulfillment, Main Store movement, branch adjustment, and per-line post-transfer balances.

NFR-09	Transactional integrity	A Branch Stock Request fulfillment is all-or-nothing across all lines and both stock locations; partial transfers are forbidden.
NFR-10	Tenant isolation	Backend authorization and queries enforce restaurant isolation and, for destination Branch Storekeepers and Managers, assigned-branch isolation. Client-side navigation is not an authorization boundary.

5. External Interface Requirements

5.1 User Interface

Single-page-app feel with a persistent left sidebar (role-filtered), light gray theme with brand-orange accents (#E8832A), and modal dialogs for POS, payment, order detail, and confirmations. On phones, the Orders/POS workspace stacks the menu above the cart and keeps checkout controls scrollable and reachable.

5.2 API Interface

REST over HTTPS with JSON bodies. The browser-facing path is `/backend/api`; the backend's direct prefix is `/api`. Interactive Swagger documentation exists at backend path `/api/docs` and must be restricted or disabled in production. Main resource groups are `/auth, /users, /restaurants, /branches, /menus, /tables, /orders, /kitchen, /payments, /inventory, /notifications, /summary, /seed`. Branch Stock Request operations are exposed beneath `/inventory/main-store/transfers`; generic Item Requests remain beneath `/inventory/requests`.

5.3 Database Interface

PostgreSQL via TypeORM repositories; connection string in `DATABASE_URL`; SSL honored when `sslmode=require` is present.

6. Assumptions & Dependencies

- A single restaurant company operates the system; multiple branches are supported.
- Kitchen staff record stock deductions/waste for profitability estimates to be accurate.
- Exports (Excel/PDF) are generated client-side (xlsx, jsPDF) and require a modern browser.
- A Main Store Storekeeper account has a restaurant assignment and no branch assignment; assigning a branch changes that user's operational identity to Branch Storekeeper.

7. Acceptance and Production Evidence

Architecture assessment is complete, but repository evidence for the full safe-refactoring, separation, automated-regression, security-hardening, migration, backup/restore, and production-verification stages is incomplete. Requirements marked in this SRS are acceptance targets, not automatic claims of production readiness. Refer to document 12 for current READY / NEEDS ATTENTION / BLOCKED status.